

Machine Learning with scikit-learn

Objectives

- ✓ The matplotlib Library
- ✓ The matplotlib Architecture
- ✓ pyplot
- ✓ Using kwargs
- ✓ Line Charts, Histograms, Bar Charts, Pie Charts, ...
- ✓ The Seaborn Library

Machine Learning

- Machine Learning in Data Analytics is the process of using historical data to train mathematical models that can predict future events, classify information, or discover hidden patterns.
- Data analytics typically includes four levels:

Level	Question Answered	Example
Descriptive	What happened?	Monthly sales report
Diagnostic	Why did it happen?	Sales dropped due to price increase
Predictive	What will happen?	Forecast next month's sales
Prescriptive	What should we do?	Recommend optimal pricing

- Machine Learning powers **Predictive** and **Prescriptive** analytics.

- Machine learning models learn relationships between:
 - **Input variables (features)** $\rightarrow X$
 - **Target variable (output)** $\rightarrow y$
 - Ex: IF income > 50,000 THEN approve loan
- ML automatically learns the rule from data.

- Role of Machine Learning in Data Analytics
 - Traditional analytics answers:
 - What happened?
 - Why did it happen?
 - Machine learning extends analytics to answer:
 - What will happen next?
 - What pattern exists in the data?
 - How can we automate decisions?

Main Types of Machine Learning in scikit-learn

A. Supervised Learning

- Model learns from labeled data.
- **Classification**
 - Predict categories.
 - Examples:
 - Spam detection
 - Disease prediction
 - Customer churn
 - Common models:
 - Logistic Regression
 - Decision Trees
 - Random Forest
 - Support Vector Machines (SVM)
 - k-Nearest Neighbors
- **Regression**
 - Predict continuous values.
 - Examples:
 - Sales forecasting
 - House price prediction
 - Common models:
 - Linear Regression
 - Ridge / Lasso
 - Random Forest Regressor

B. Unsupervised Learning

- Model finds hidden patterns without labels.
- **Clustering**
 - K-Means
 - DBSCAN
 - Hierarchical Clustering
- **Dimensionality Reduction**
 - PCA (Principal Component Analysis)

- scikit-learn is an open-source Python library built on:
 - NumPy
 - SciPy
 - Matplotlib
- Provides **simple, efficient tools** for:
 - Classification
 - Regression
 - Clustering
 - Dimensionality reduction
 - Model selection
 - Data preprocessing
- It is especially popular in:
 - Academic teaching
 - Business analytics
 - Rapid prototyping
 - Industry ML pipelines

Typical Machine Learning Workflow in Data Analytics

- Step 1: Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

- Step 2: Split Data

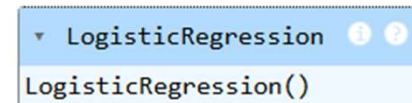
```
from sklearn.datasets import load_iris
```

```
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

- Step 3: Train Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```



- Step 4: Make Predictions

```
predictions = model.predict(X_test)
```

- Step 5: Evaluate Model

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, predictions)
```

```
0.9666666666666667
```

- **Why scikit-learn is Important in Data Analytics**

- Simple and consistent API
- Works well with Pandas
- Excellent documentation
- Production-ready algorithms
- Ideal for structured/tabular data

- It is **not** primarily for:

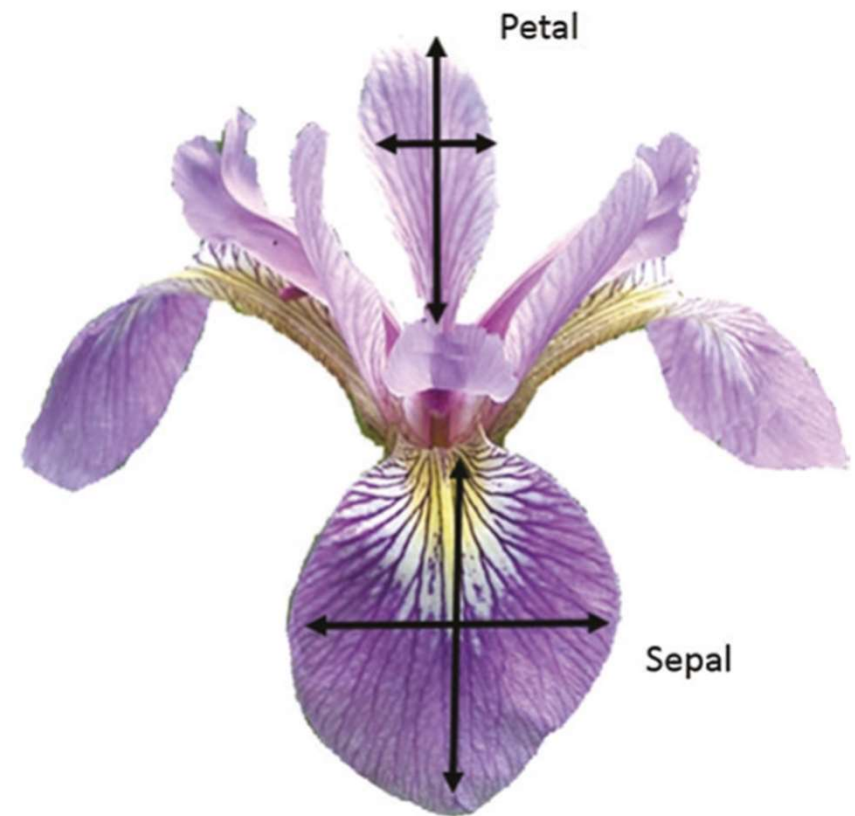
- Deep learning (use TensorFlow / PyTorch instead)
- Very large distributed systems

Real Business Example

- Problem:
Will a customer cancel their subscription?
- Steps:
 - Use past customer behavior data
 - Train a classification model
 - Predict churn probability
 - Take preventive action for high-risk customers
- This transforms raw data into **actionable insight**.

The Iris Flower Dataset

The Iris Flower Dataset was first introduced by Ronald Fisher in 1936 and is also known as the Anderson Iris Dataset, named after the researcher who collected the measurements. It contains data on three iris species—Iris Sentosa, Iris virginica, and Iris versicolor—including the length and width of their sepals and petals.



Iris versicolor and the petal and sepal width and length

- Loaded all the data and metadata concerning the Iris Dataset into the iris variable.

```
from sklearn import datasets  
iris = datasets.load_iris()
```

```
#150 elements, each containing four numeric values:  
#the size of sepals and petals respectively  
iris.data
```

```
[5.8, 2.6, 4. , 1.2],  
[5. , 2.3, 3.3, 1. ],  
[5.6, 2.7, 4.2, 1.3],  
[5.7, 3. , 4.2, 1.2],  
[5.7, 2.9, 4.2, 1.3],  
[6.2, 2.9, 4.3, 1.3],  
[5.1, 2.5, 3. , 1.1],  
[5.7, 2.8, 4.1, 1.3],  
[6.3, 3.3, 6. , 2.5],  
[5.8, 2.7, 5.1, 1.9],  
[7.1, 3. , 5.9, 2.1],  
[6.3, 2.9, 5.6, 1.8],  
[6.5, 3. , 5.8, 2.2],
```

- To determine instead what kind of flower belongs to each item, refer to the target attribute.

```
iris.target
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
#0: setosa - 1: versicolor - 2: virginica
```

```
iris.target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

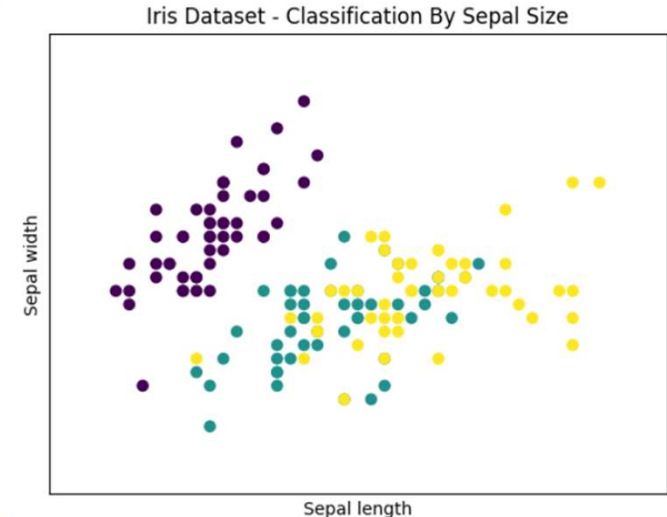
- Use the matplotlib library.
- Create a scatterplot that displays the three different species in three different colors. The x-axis will represent the length of the **sepal**, while the y-axis will represent the width of the sepal.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn import datasets

iris = datasets.load_iris()
x = iris.data[:,0] #X-Axis - sepal length
y = iris.data[:,1] #Y-Axis - sepal width
species = iris.target #species

x_min, x_max = x.min() - .5, x.max() + .5
y_min, y_max = y.min() - .5, y.max() + .5

#SCATTERPLOT
plt.figure()
plt.title('Iris Dataset - Classification By Sepal Size')
plt.scatter(x,y, c=species)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())
plt.show()
```



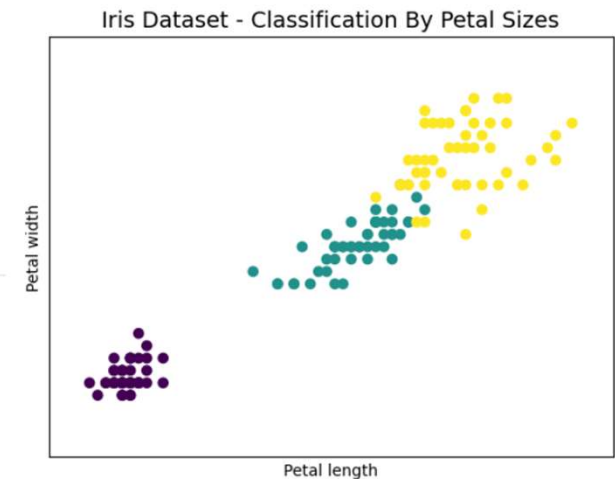
- Using the other two variables—that is, the measure of the length and width of the **petal**.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn import datasets

iris = datasets.load_iris()
x = iris.data[:,2] #X-Axis - petal length
y = iris.data[:,3] #Y-Axis - petal width
species = iris.target #species

x_min, x_max = x.min() - .5, x.max() + .5
y_min, y_max = y.min() - .5, y.max() + .5

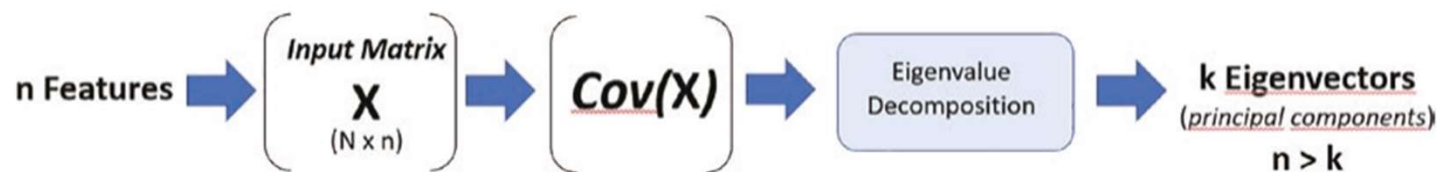
#SCATTERPLOT
plt.figure()
plt.title('Iris Dataset - Classification By Petal Sizes', size = 14)
plt.scatter(x,y, c=species)
plt.xlabel('Petal length')
plt.ylabel('Petal width')
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())
plt.show()
```



The PCA Decomposition

- When analyzing the three iris species using four measurements (petal and sepal length and width), visualizing all four dimensions at once becomes difficult—even a 3D scatterplot is insufficient.
- To solve this, **Principal Component Analysis (PCA)** is used. PCA reduces the number of dimensions while preserving most of the important information. It transforms the original variables into new ones called *principal components*, which capture the maximum variance in the dataset.
- PCA is typically applied before a machine learning algorithm to simplify the model. It identifies and removes features that contribute little new information (i.e., redundant variables), reducing computational complexity and improving efficiency.

- Mathematically, PCA works by:
 - Starting with the data matrix (X) (observations $\times$ features).
 - Computing the covariance matrix.
 - Calculating its eigenvectors and eigenvalues.
 - Selecting the top k eigenvectors corresponding to the largest eigenvalues.
- These selected eigenvectors form the principal components, representing a lower-dimensional version of the original dataset while retaining most of its information.



The processes involved in the PCA technique

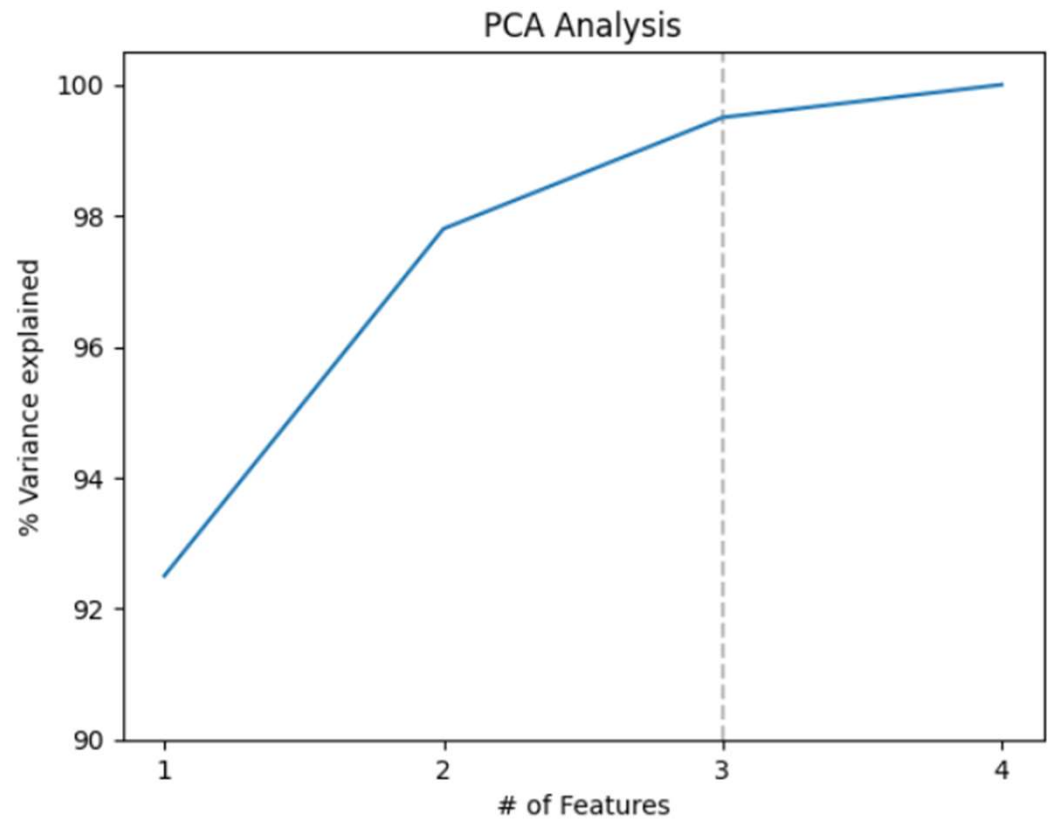
This section applies Principal Component Analysis (PCA) to the Iris Dataset, which contains four features. The goal is to determine whether the number of features can be reduced while still preserving most of the information needed for an effective machine learning model.

```
from sklearn.decomposition import PCA
import numpy as np
covar_matrix = PCA(n_components=4)
covar_matrix.fit(iris.data)
variance = covar_matrix.explained_variance_ratio_
var = np.cumsum(np.round(variance, decimals=3)*100)
var
```

```
array([ 92.5,  97.8,  99.5, 100. ])
```

- The code imports the **PCA() function** from `sklearn.decomposition` and applies it to all four features of the dataset by setting **n_components = 4**. It computes the covariance matrix and performs eigenvalue decomposition using the **fit()** method.
- Next, it evaluates how much each feature contributes to the total variance. The explained variance is extracted, and NumPy's **cumsum() function** is used to calculate the cumulative percentage of variance explained by the components.
- The results show that even reducing the dataset to just one principal component would retain about 92% of the total information, suggesting that a single feature may be nearly sufficient for building an effective machine learning model. The variance contribution can also be visualized graphically using Matplotlib.

```
plt.ylabel('% Variance explained')
plt.xlabel('# of Features')
plt.title('PCA Analysis')
plt.ylim(90, 100.5)
plt.xticks([0, 1, 2, 3], [1,2,3,4])
plt.axvline(2, linestyle='--', c='#bbbbbb' )
plt.plot(var)
plt.show()
```



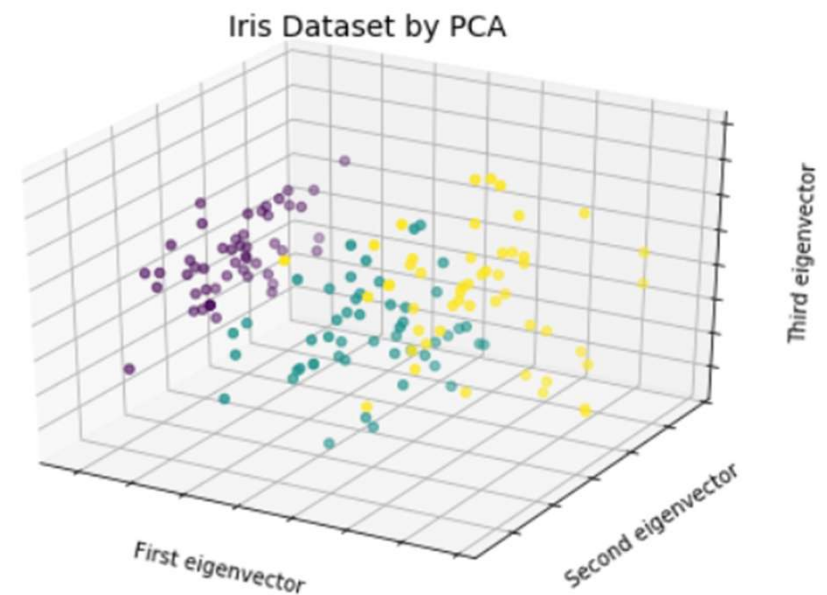
The contribution of each feature to the total variance

- In addition, in order to visualize the new values, you will use a 3D scatterplot using the **mpl_toolkits.mplot3d module** of matplotlib.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import datasets
from sklearn.decomposition import PCA

iris = datasets.load_iris()
#x = iris.data[:,1] #X-Axis - sepal width
#y = iris.data[:,2] #Y-Axis - petal length
species = iris.target #species
x_reduced = PCA(n_components=3).fit_transform(iris.data)

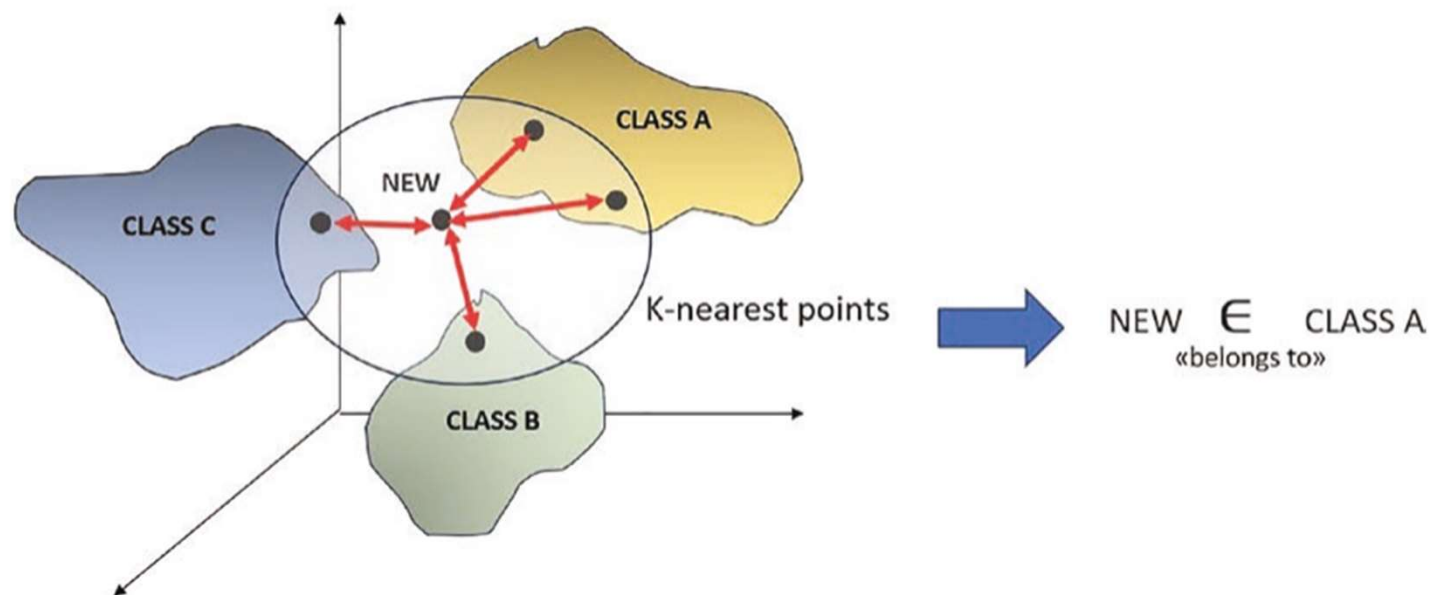
#SCATTERPLOT 3D
fig = plt.figure()
ax = Axes3D(fig)
ax.set_title('Iris Dataset by PCA', size = 14)
ax.scatter(x_reduced[:,0],x_reduced[:,1],x_reduced[:,2], c=species)
ax.set_xlabel('First eigenvector')
ax.set_ylabel('Second eigenvector')
ax.set_zlabel('Third eigenvector')
ax.xaxis.set_ticklabels(())
ax.yaxis.set_ticklabels(())
ax.zaxis.set_ticklabels(())
plt.show()
```



K-Nearest Neighbors Classifier

- To perform classification with scikit-learn, a classifier is required. The task is to assign a new iris flower measurement to one of three species. A simple and widely used method is the **K-Nearest Neighbors (KNN)** algorithm.
- KNN works by finding the training samples that are closest to a new data point in the multidimensional feature space. It calculates distances using metrics such as Euclidean, Manhattan, or Minkowski (though Euclidean distance may be less effective in high-dimensional spaces).
- After computing distances, the algorithm selects the **K closest neighbors**. For classification, the new sample is assigned to the class that appears most frequently among these K nearest neighbors. Despite its simplicity, KNN is widely used for classification, regression, and outlier detection.

- *The KNN mechanism of classification*

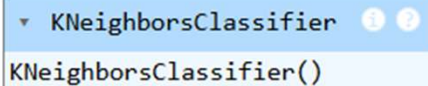


- Before building a classifier, it is essential to separate the dataset into a **training set** and a **testing set**. The training set is used to teach the algorithm, while the testing set is used to evaluate its performance. Using the same data for both would lead to unreliable results.
- For the Iris Dataset, the data must first be randomly shuffled because it is originally ordered by species. This prevents bias in the split. NumPy's **random.permutation()** function is used to mix the 150 observations.
- After shuffling, **140 samples** are assigned to the **training set** and the remaining **10 samples** are used as the **test set** for validation.

```
import numpy as np
from sklearn import datasets
np.random.seed(0)
iris = datasets.load_iris()
x = iris.data
y = iris.target
i = np.random.permutation(len(iris.data))
x_train = x[i[:-10]]
y_train = y[i[:-10]]
x_test = x[i[-10:]]
y_test = y[i[-10:]]
```

- Apply the K-Nearest Neighbor algorithm.
- Import the **KNeighborsClassifier**, call the constructor of the classifier, and then train it with the **fit() function**.

```
from sklearn.neighbors import KNeighborsClassifier  
knn = KNeighborsClassifier()  
knn.fit(x_train, y_train)
```



```
▼ KNeighborsClassifier ⓘ ⓘ  
KNeighborsClassifier()
```

After training the KNN classifier on 140 observations, its performance is evaluated using the 10 test samples. The `predict()` function is applied to the **trained knn model** to generate species predictions for the test set. These predicted labels are then compared with the actual labels in **y_test** to assess how accurately the model classifies the iris species.

```
knn.predict(x_test)
```

```
array([1, 2, 1, 0, 0, 0, 2, 1, 2, 0])
```

```
y_test
```

```
array([1, 1, 1, 0, 0, 0, 2, 1, 2, 0])
```

- The results show a **10% classification error**, meaning the model misclassified 1 out of the 10 test samples.
- This performance can be visualized using **decision boundaries** plotted on a 2D scatterplot of the sepal features (sepal length vs. sepal width). The decision boundaries divide the feature space into regions corresponding to different predicted species, allowing you to clearly see how the KNN classifier separates the classes and where misclassifications occur.

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier

iris = datasets.load_iris()
x = iris.data[:, :2] #X-Axis - sepal length-width
y = iris.target      #Y-Axis - species

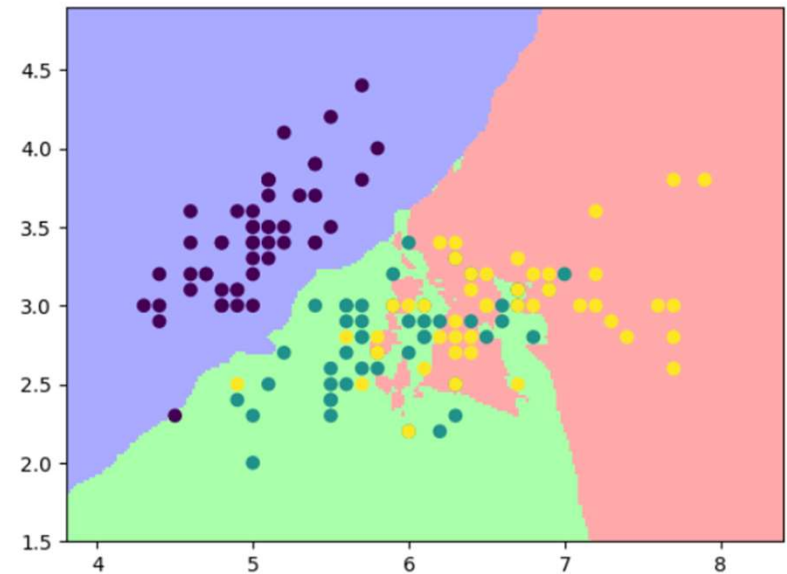
x_min, x_max = x[:, 0].min() - .5, x[:, 0].max() + .5
y_min, y_max = x[:, 1].min() - .5, x[:, 1].max() + .5

#MESH
cmap_light = ListedColormap(['#AAAAFF', '#AAFFAA', '#FFAAAA'])
h = .02
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
knn = KNeighborsClassifier()
knn.fit(x, y)
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure()
plt.pcolormesh(xx, yy, Z, cmap=cmap_light)

#Plot the training points
plt.scatter(x[:, 0], x[:, 1], c=y)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.show()

```

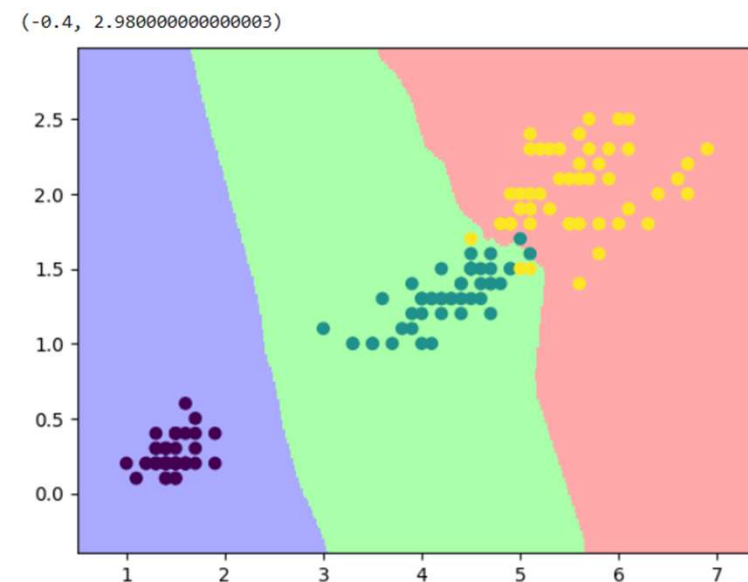


The three decision boundaries are represented by three different colors

```

#do the same thing considering the size of the petals.
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier
iris = datasets.load_iris()
x = iris.data[:,2:4] #X-Axis - petals length-width
y = iris.target #Y-Axis - species
x_min, x_max = x[:,0].min() - .5, x[:,0].max() + .5
y_min, y_max = x[:,1].min() - .5, x[:,1].max() + .5
#MESH
cmap_light = ListedColormap(['#AAAAFF', '#AAFFAA', '#FFAAAA'])
h = .02
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
knn = KNeighborsClassifier()
knn.fit(x,y)
Z = knn.predict(np.c_[xx.ravel(),yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure()
plt.pcolormesh(xx,yy,Z,cmap=cmap_light, shading='auto')
#Plot the training points
plt.scatter(x[:,0],x[:,1],c=y)
plt.xlim(xx.min(),xx.max())
plt.ylim(yy.min(),yy.max())

```



The decision boundaries on a 2D scatterplot describing the petal sizes

Diabetes Dataset

- To load this dataset in scikit-learn, you import the `datasets` module and use the **`load_diabetes()` function**, assigning the result to a variable (e.g., `diabetes`) for further analysis.
- The Diabetes dataset includes physiological measurements from 442 patients, along with a target variable representing disease progression one year later.
- The dataset contains 10 input features:
 - Age
 - Sex
 - Body Mass Index (BMI)
 - Blood Pressure
 - S1–S6 (six blood serum measurements)

```
from sklearn import datasets
diabetes = datasets.load_diabetes()
```

```
#look at the ten values for the first patient.
diabetes.data[0]
```

```
array([ 0.03807591,  0.05068012,  0.06169621,  0.02187239, -0.0442235 ,
        -0.03482076, -0.04340085, -0.00259226,  0.01990749, -0.01764613])
```

- The feature values in the dataset have been preprocessed before being provided. Each of the ten variables was:
 - **Mean-centered** (the average value was subtracted), and
 - **Scaled** by dividing by the standard deviation multiplied by the square root of the number of samples.
- As a result of this normalization, the sum of the squares of the values in each column equals approximately 1. For example, if you compute the sum of squared values for the **age** feature, you will obtain a value very close to 1, confirming the scaling process.

```
np.sum(diabetes.data[:,0]**2)
```

```
np.float64(1.0000000000000009)
```


- Although the feature values are normalized and less intuitive to interpret, they still represent the original ten physiological characteristics and retain all their statistical information.
- The measure of disease progression—used as the prediction target—is available through the dataset's **target attribute**, which contains the values the predictive model aims to estimate.

diabetes.target

```
array([151., 75., 141., 206., 135., 97., 138., 63., 110., 310., 101.,
       69., 179., 185., 118., 171., 166., 144., 97., 168., 68., 49.,
       68., 245., 184., 202., 137., 85., 131., 283., 129., 59., 341.,
       87., 65., 102., 265., 276., 252., 90., 100., 55., 61., 92.,
       259., 53., 190., 142., 75., 142., 155., 225., 59., 104., 182.,
       128., 52., 37., 170., 170., 61., 144., 52., 128., 71., 163.,
       150., 97., 160., 178., 48., 270., 202., 111., 85., 42., 170.,
       200., 252., 113., 143., 51., 52., 210., 65., 141., 55., 134.,
       42., 111., 98., 164., 48., 96., 90., 162., 150., 279., 92.,
       83., 128., 102., 302., 198., 95., 53., 134., 144., 232., 81.,
       104., 59., 246., 297., 258., 229., 275., 281., 179., 200., 200.,
       173., 180., 84., 121., 161., 99., 109., 115., 268., 274., 158.,
       107., 83., 103., 272., 85., 280., 336., 281., 118., 317., 235.,
       60., 174., 259., 178., 128., 96., 126., 288., 88., 292., 71.,
       197., 186., 25., 84., 96., 195., 53., 217., 172., 131., 214.,
       59., 70., 220., 268., 152., 47., 74., 295., 101., 151., 127.,
       237., 225., 81., 151., 107., 64., 138., 185., 265., 101., 137.,
```

obtain a series of 442 integer values between 25 and 346.

Linear Regression: The Least Square Regression

- Linear regression builds a linear model from the training data to predict a target variable. In its simplest form, it represents a straight line defined by two parameters: slope and intercept. These parameters are calculated by minimizing the **sum of squared residuals** (the differences between predicted and actual values).
- The model can be expressed as: $y = ax + c$
where:
 - (x) = input (training data)
 - (y) = target value
 - (a) = slope
 - (c) = intercept
- In scikit-learn, linear regression is implemented by importing the **linear_model module** and creating a model instance using the **LinearRegression()** constructor (e.g., linreg).

- To practice with an example of linear regression, you can use the diabetes dataset described earlier. First you need to break the **442 patients** into a **training set** (composed of the first 422 patients) and a **test set** (the last **20 patients**).

```
from sklearn import linear_model  
linreg = linear_model.LinearRegression()
```

```
from sklearn import datasets  
diabetes = datasets.load_diabetes()  
x_train = diabetes.data[:-20]  
y_train = diabetes.target[:-20]  
x_test = diabetes.data[-20:]  
y_test = diabetes.target[-20:]
```

```
#apply the training set to the predictive model through  
#the use of the fit() function.  
linreg.fit(x_train,y_train)
```

▼ LinearRegression ⓘ ?
LinearRegression()

```
#Once the model is trained, you can get the ten b coefficients  
#calculated for each physiological variable,  
#using the coef_ attribute of the predictive model.  
linreg.coef_
```

```
array([ 3.06094248e-01, -2.37635570e+02,  5.10538048e+02,  3.27729878e+02,  
       -8.14111926e+02,  4.92799595e+02,  1.02841240e+02,  1.84603496e+02,  
        7.43509388e+02,  7.60966464e+01])
```

- Apply the test set to the **linreg** prediction model, you will get a series of targets to be compared with the values actually observed.

```
linreg.predict(x_test)
```

```
array([197.61898486, 155.44031962, 172.88875144, 111.53270645,  
       164.79397301, 131.06765869, 259.12441219, 100.47873746,  
       117.06005372, 124.30261597, 218.36868146, 61.19581944,  
       132.24837933, 120.33293546, 52.54513009, 194.03746764,  
       102.5756431 , 123.56778709, 211.03465323, 52.60221696])
```

```
y_test
```

```
array([233., 91., 111., 152., 120., 67., 310., 94., 183., 66., 173.,  
       72., 49., 64., 48., 178., 104., 132., 220., 57.])
```

- a good indicator of which prediction should be perfect is the *variance*. The closer the variance is to 1, the more perfect the prediction.

```
linreg.score(x_test, y_test)
```

```
0.585085427447195
```

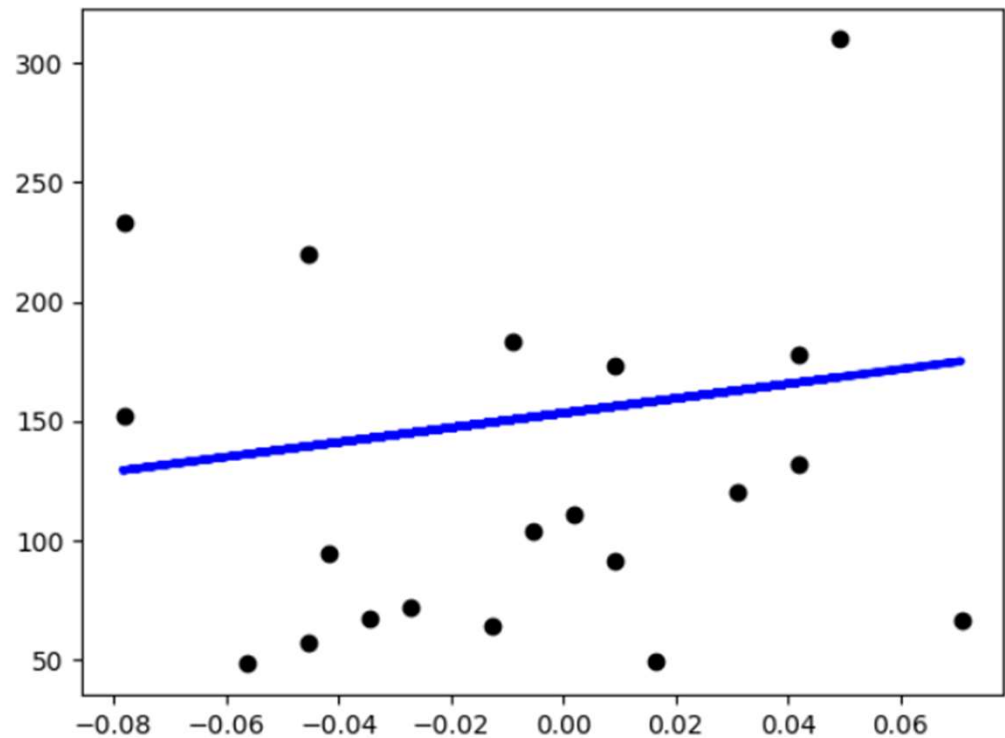
- You now begin linear regression using only **one physiological feature**, such as **age**, as the predictor variable.
- In this case, the model attempts to explain disease progression using age alone. This represents a **simple linear regression**, where:
 - x = age
 - y = disease progression
- The algorithm estimates the slope and intercept that best fit the data.
- This approach allows you to evaluate how strongly age alone influences disease progression before extending the model to include multiple features.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn import datasets
diabetes = datasets.load_diabetes()
x_train = diabetes.data[:-20]
y_train = diabetes.target[:-20]
x_test = diabetes.data[-20:]
y_test = diabetes.target[-20:]

x0_test = x_test[:,0]
x0_train = x_train[:,0]
x0_test = x0_test[:,np.newaxis]
x0_train = x0_train[:,np.newaxis]
linreg = linear_model.LinearRegression()
linreg.fit(x0_train, y_train)
y = linreg.predict(x0_test)
plt.scatter(x0_test, y_test, color='k')
plt.plot(x0_test, y, color='b', linewidth=3)
plt.show()

```

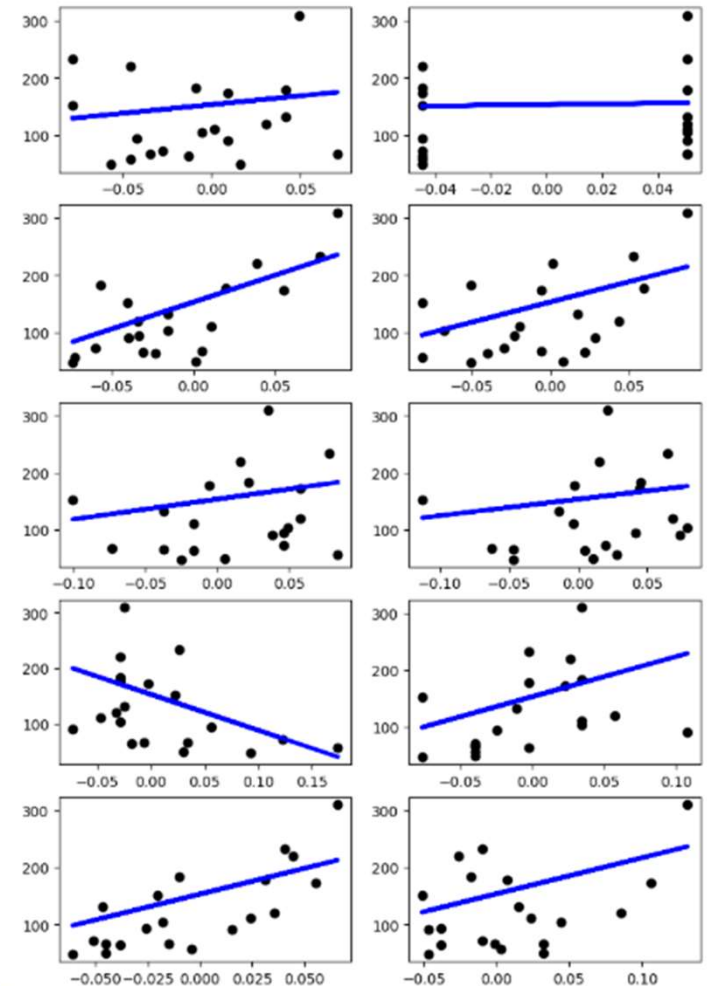


the line representing the linear correlation between the ages of the patients and the disease progression.

- To have a more complete picture of all the training set, you can create a linear regression for every physiological feature, creating ten models and seeing the result for each of them through a linear chart.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn import datasets
diabetes = datasets.load_diabetes()
x_train = diabetes.data[:-20]
y_train = diabetes.target[:-20]
x_test = diabetes.data[-20:]
y_test = diabetes.target[-20:]

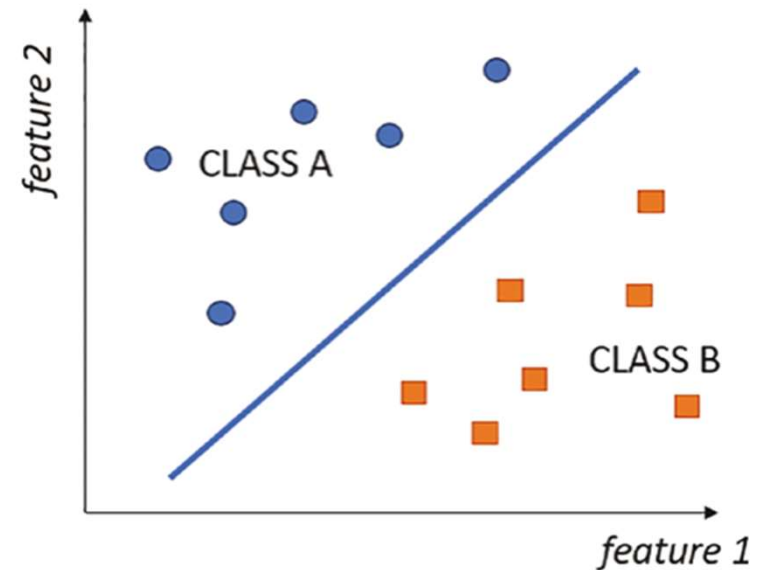
plt.figure(figsize=(8,12))
for f in range(0,10):
    xi_test = x_test[:,f]
    xi_train = x_train[:,f]
    xi_test = xi_test[:,np.newaxis]
    xi_train = xi_train[:,np.newaxis]
    linreg.fit(xi_train, y_train)
    y = linreg.predict(xi_test)
    plt.subplot(5,2,f+1)
    plt.scatter(xi_test,y_test,color='k')
    plt.plot(xi_test, y, color='b', linewidth=3)
plt.show()
```



Support Vector Machines (SVMs)

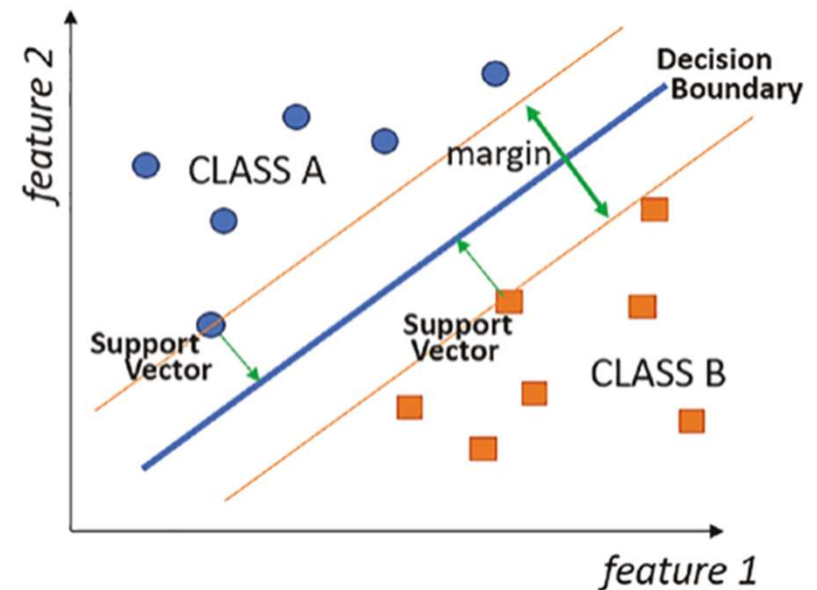
- SVMs are primarily **binary classification models** designed to distinguish between two classes. During training, they map data into a multidimensional **decision space** and construct a **decision boundary** that separates the classes.
- In the **linear case**, the boundary is a straight line (2D) or a plane (3D).
- In more complex cases, the boundary becomes a curved surface capable of handling nonlinear separations.
- Their main goal is to accurately classify new observations by determining which side of the decision boundary they fall on.

- Also for SVM, the data points are distributed in a multidimensional space, where the values of a specific feature are distributed on each axis. The purpose of SVM is to find a hyperplane separating the space between the two classes



SVM divides the decision space into two areas, each for each class, defining a straight line (decision boundary)

- The straight line that must be defined must not limit itself to separating the points of the two classes from each other, but must respond to particular requirements.
- SVM finds the points that are closest to the dividing line. These are called “support vectors” and they give the algorithm its name



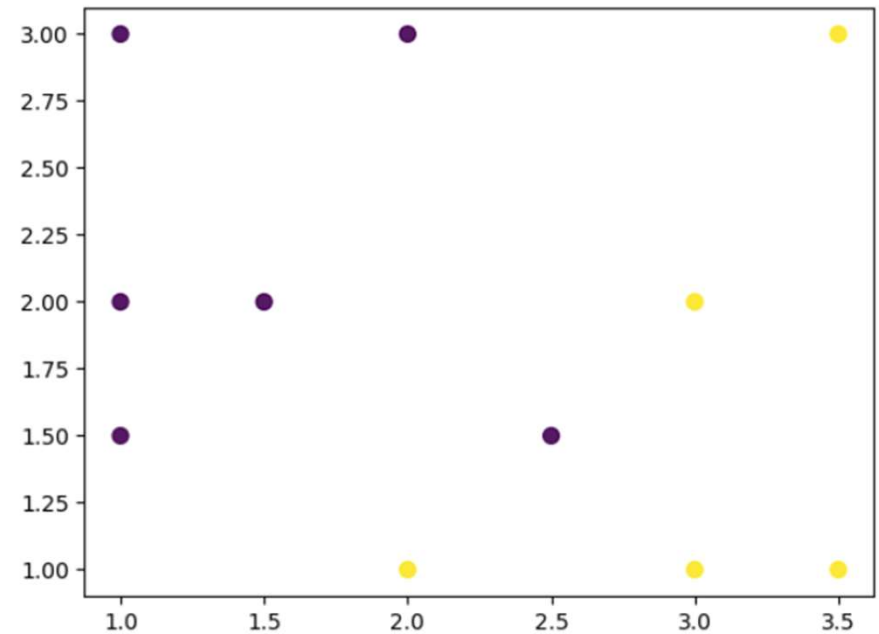
SVM optimizes the decision boundary, maximizing the margin between support vector points

Support Vector Classification (SVC)

- To understand SVM more clearly, consider the simplest scenario: a **linear 2D case**, where the decision boundary is a straight line dividing the space into two regions.
- In this example, the training set contains **11 data points**, each described by **two attributes** with values ranging from 0 to 4. These feature values are stored in a NumPy array called `x`.
- Each point belongs to one of two classes, labeled **0 or 1**, and these class labels are stored in another array called `y`.
- Using this setup, SVM will attempt to find the optimal straight line that separates the two classes while maximizing the margin between them.

- Visualize distribution of these points in space with a scatterplot, which will then be defined as a decision space

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
x = np.array([[1,3],[1,2],[1,1.5],[1.5,2],[2,3],[2.5,1.5],
              [2,1],[3,1],[3,2],[3.5,1],[3.5,3]])
y = [0]*6 + [1]*5
plt.scatter(x[:,0],x[:,1],c=y, s=50, alpha=0.9)
plt.show()
```

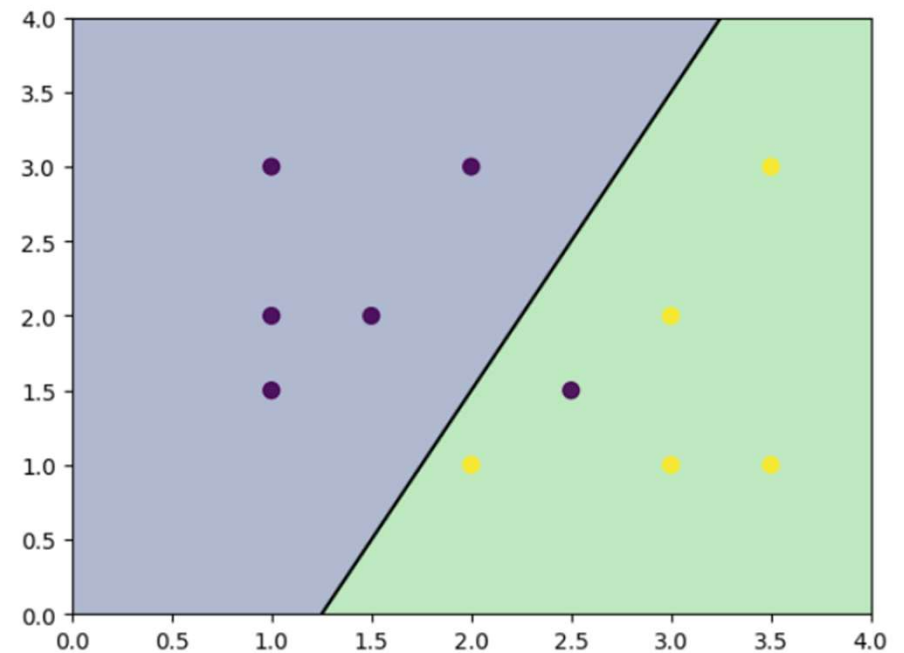


- After defining the training set, you apply the **Support Vector Classification (SVC)** algorithm to build a linear classifier. The goal of SVC is to create a **decision boundary** (a straight line in 2D) that separates the two classes while **maximizing the margin**, meaning it maximizes the distance from the closest training points (the support vectors).
- To implement this:
 - Create the model using the **SVC() constructor** and set the kernel='linear' parameter (the kernel defines the type of decision boundary; here it is linear).
 - Train the model using the **fit() function** with the training data.
 - After training, compute the decision boundary using the **decision_function()**.
 - Finally, **plot** a scatterplot of the data and color the two regions of the decision space differently to visualize how the boundary separates the classes.
- This visualization clearly shows how SVC divides the space and maximizes the margin between the two groups.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
x = np.array([[1,3],[1,2],[1,1.5],[1.5,2],[2,3],[2.5,1.5],
              [2,1],[3,1],[3,2],[3.5,1],[3.5,3]])
y = [0]*6 + [1]*5
svc = svm.SVC(kernel='linear').fit(x,y)
X,Y = np.mgrid[0:4:200j,0:4:200j]
Z = svc.decision_function(np.c_[X.ravel(),Y.ravel()])
Z = Z.reshape(X.shape)
plt.contourf(X,Y,Z > 0, alpha=0.4)
plt.contour(X,Y,Z,colors=['k'], linestyles=['-'], levels=[0])
plt.scatter(x[:,0],x[:,1],c=y, s=50, alpha=0.9)
plt.show()

```



- Once the SVC model is trained, predictions are straightforward.
- **Graphically**, a new observation is classified based on which side of the decision boundary it falls on. Its position in the decision space determines whether it belongs to class 0 or class 1.
- **Programmatically**, you use the `predict()` function, which returns the class label directly—**0** (e.g., blue class) or **1** (e.g., red class).
- Thus, classification is simply a matter of determining on which side of the boundary the new data point lies.

```
svc.predict([[1.5,2.5]])
```

```
array([0])
```

```
svc.predict([[2.5,1]])
```

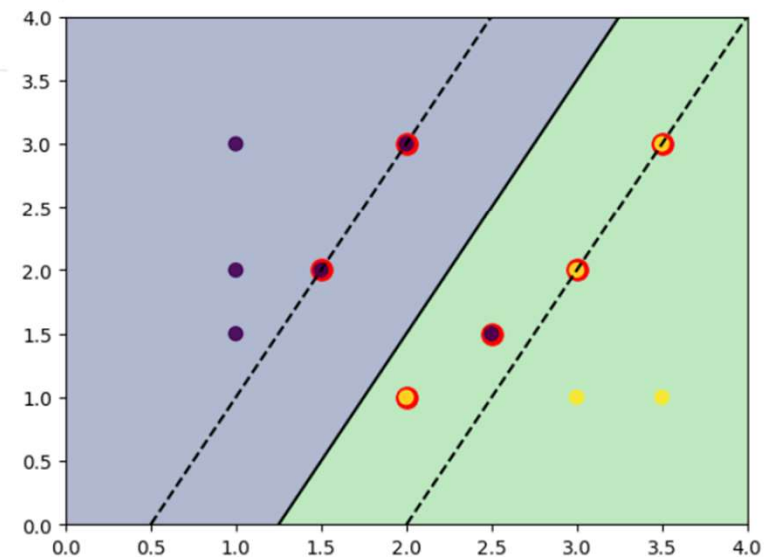
```
array([1])
```


- In the SVC algorithm, **regularization** is controlled by the parameter **C**.
- A **small value of C** applies **stronger regularization**, allowing a wider margin and considering many (or even all) nearby observations when determining the separation boundary. This may tolerate some misclassifications to achieve better generalization.
- A **large value of C** applies **weaker regularization**, focusing more strictly on correctly classifying points closest to the boundary, which can lead to a narrower margin.
- By default, **C = 1**.
- After training, the points that directly influence the margin—called **support vectors**—can be identified using the `support_vectors_` attribute of the model. These are the observations that define the position of the decision boundary.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
x = np.array([[1,3],[1,2],[1,1.5],[1.5,2],[2,3],[2.5,1.5],
              [2,1],[3,1],[3,2],[3.5,1],[3.5,3]])
y = [0]*6 + [1]*5
svc = svm.SVC(kernel='linear', C=1).fit(x,y) # Ho tolto C=1
X,Y = np.mgrid[0:4:200j,0:4:200j]
Z = svc.decision_function(np.c_[X.ravel(),Y.ravel()])
Z = Z.reshape(X.shape)
plt.contourf(X,Y,Z > 0, alpha=0.4)
plt.contour(X,Y,Z,colors=['k','k','k'], linestyles=['--','-','-'], levels=[-1,0,1])
plt.scatter(svc.support_vectors_[0],svc.support_vectors_[1],s=120, facecolors='r')
plt.scatter(x[:,0],x[:,1],c=y, s=50, alpha=0.9)
plt.show()

```



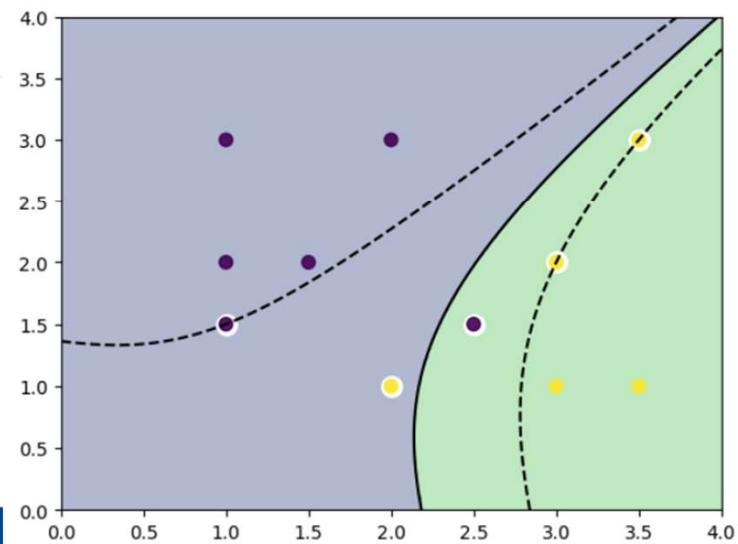
Nonlinear SVC

- Until now, the SVC model used a **linear kernel**, producing a straight decision boundary. However, SVC can also create **nonlinear boundaries**—such as curves in 2D or curved surfaces in higher dimensions—while still maximizing the margin between the closest points (support vectors).
- Using a **polynomial kernel**, the algorithm can generate a curved decision boundary. The complexity of this curve is controlled by the **degree** parameter, which sets the polynomial degree. The **C parameter** still controls regularization (with default value 1).
- To apply this model:
 - Use `SVC(kernel='poly', degree=3, C=1)`
 - Then train the model with `fit()` on the training data.
- This will produce a third-degree polynomial decision boundary that separates the two classes while maintaining margin maximization.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
x = np.array([[1,3],[1,2],[1,1.5],[1.5,2],[2,3],[2.5,1.5],
              [2,1],[3,1],[3,2],[3.5,1],[3.5,3]])
y = [0]*6 + [1]*5
svc = svm.SVC(kernel='poly', C=1, degree=3).fit(x,y) # Ho tolto C=1
X,Y = np.mgrid[0:4:200j,0:4:200j]
Z = svc.decision_function(np.c_[X.ravel(),Y.ravel()])
Z = Z.reshape(X.shape)
plt.contourf(X,Y,Z > 0, alpha=0.4)
plt.contour(X,Y,Z,colors=['k','k','k'], linestyles=['--','-','-'], levels=[-1,0,1])
plt.scatter(svc.support_vectors_[0],svc.support_vectors_[1],s=120, facecolors='w')
plt.scatter(x[:,0],x[:,1],c=y, s=50, alpha=0.9)
plt.show()

```



Plotting Different SVM Classifiers Using the Iris Dataset

- This section applies **Support Vector Classification (SVC)** to a more complex dataset: the Iris Dataset, which contains **three classes** (three flower species), instead of just two.
- Unlike the earlier binary example, SVC now performs **multiclass classification**, creating multiple decision boundaries that may intersect. In 2D, these boundaries divide the space into several regions; in 3D, they partition the decision volume into multiple areas corresponding to each species.
- With a **linear kernel**, the model produces linear decision boundaries (intersecting hyperplanes).
- With **nonlinear kernels** (such as polynomial or Gaussian RBF), the model generates curved, more flexible boundaries.
- The shape and flexibility of these boundaries depend on the chosen kernel and its parameters, allowing the model to better adapt to more complex class distributions.

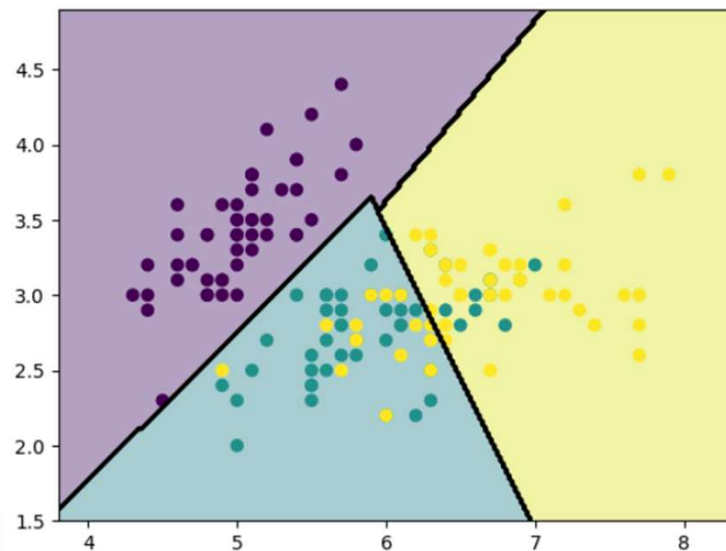
```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets

iris = datasets.load_iris()
x = iris.data[:, :2]
y = iris.target
h = .05
svc = svm.SVC(kernel='linear', C=1.0).fit(x, y)
x_min, x_max = x[:, 0].min() - .5, x[:, 0].max() + .5
y_min, y_max = x[:, 1].min() - .5, x[:, 1].max() + .5
h = .02
X, Y = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

Z = svc.predict(np.c_[X.ravel(), Y.ravel()])
Z = Z.reshape(X.shape)
plt.contourf(X, Y, Z, alpha=0.4)
plt.contour(X, Y, Z, colors='k')
plt.scatter(x[:, 0], x[:, 1], c=y)
plt.show()

```



Support Vector Regression (SVR)

- **Support Vector Regression (SVR)** is an extension of SVC for solving **regression** problems instead of classification.
- Like SVC, SVR does not rely on the entire training dataset. Instead, the model depends only on a **subset of training points** (called support vectors), which are the observations most relevant to defining the regression function.
- In this example, SVR is applied to the **Diabetes dataset**, using only the **third physiological feature** as the predictor. Three regression models are built:
- **One linear SVR**, which produces a straight-line fit (similar to standard linear regression).
- **Two nonlinear polynomial SVRs**, using second- and third-degree polynomials to produce curved regression functions.
- The SVR() function is very similar in structure to SVC(). One important detail is that the test data must be **sorted in ascending order** before plotting, ensuring a clear and properly drawn regression curve.


```

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn import datasets
diabetes = datasets.load_diabetes()
x_train = diabetes.data[:-20]
y_train = diabetes.target[:-20]
x_test = diabetes.data[-20:]
y_test = diabetes.target[-20:]
x0_test = x_test[:,2]
x0_train = x_train[:,2]
x0_test = x0_test[:,np.newaxis]
x0_train = x0_train[:,np.newaxis]
x0_test.sort(axis=0)
x0_train = x0_train*100
x0_train = x0_train*100
svr = svm.SVR(kernel='linear',C=1000)
svr2 = svm.SVR(kernel='poly',C=1000,degree=2)
svr3 = svm.SVR(kernel='poly',C=1000,degree=3)
svr.fit(x0_train,y_train)
svr2.fit(x0_train,y_train)
svr3.fit(x0_train,y_train)
y = svr.predict(x0_test)
y2 = svr2.predict(x0_test)
y3 = svr3.predict(x0_test)
plt.scatter(x0_test,y_test,color='k')
plt.plot(x0_test,y,color='b')
plt.plot(x0_test,y2,color='r')
plt.plot(x0_test,y3,color='g')

```

